Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 3

Part A (To be referred by students)

Looping & Strings

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp3)

Aim: Write a program to demonstrate Looping Statements (while, for, nested loops, break, continue, else with loop), Pass and string functions.

Problem Statement: Write Python program to

1. WAP to print the sum of the first n natural numbers using a for loop
2. Count the number of even and odd numbers from a series of numbers P to Q using a for loop.
3. Write a program to check whether the entered number is prime or not (make use of break)
4. Display the Fibonacci sequence up to the nth term, where n is provided by the user (use a while loop).
5. Print the following pattern using a nested loop.

```
  *
 ***
*****
```

6. WAP to print the following pattern

```
*****        1          *
****         12         **
***          123        ***
**           1234       **
*            12345      *
```

7. Write a Python program that iterates the integers from 1 to 50. For multiples of three print "Fizz" instead of the number and for multiples of five print "Buzz". For numbers that are multiples of three and five, print "FizzBuzz".
   Sample Output :
   fizzbuzz
   1
   2
   fizz
   4
   buzz
8. Demonstrate 5 string operations using string function
9. Demonstrate using the continue statement in a loop
10. Demonstrate use of Pass in loop and else with loop

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Topic covered: Loops (for, while) else with loop, pass & string functions

Learning Objective: Learner would be able to

1. Analyze the scenario to write script applying looping.
2. Solve the problems using looping statements
3. Understand and use pass, else, continue & break
4. Process string and perform various operations on string

Theory:

- **The for loop:**

  The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. With for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Syntax of for Loop

```
for val in sequence:
    loop body
```

Here, val is the variable that takes the value of the item inside the sequence on each iteration.

Loop continues until we reach the last item in the sequence. The body of for loop is separated from the rest of the code using indentation.

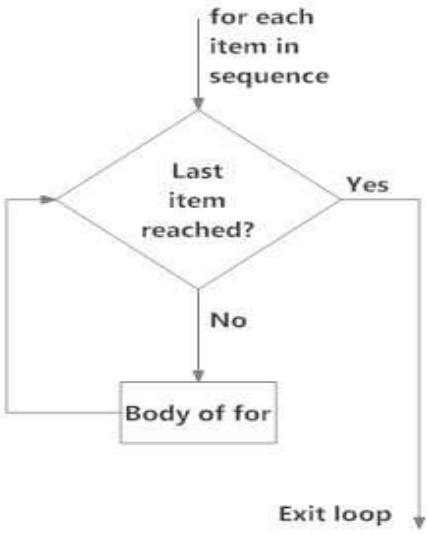


Fig: operation of for loop

The for loop does not require an indexing variable to set beforehand, as the for command itself allows for this.

```
# Print each fruit in a fruit list:
fruits = ["apple", "banana", "cherry"]
for x in fruits:
```

```
print(x)
```

Output:

apple

banana

cherry

- **The while loop:**

  The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects. With for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

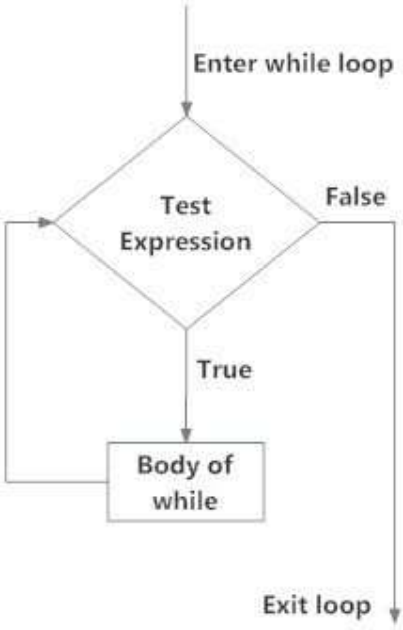
Syntax of while Loop	
while test_expression: **loop body**	
In the while loop, test expression is checked first. The body of the loop is entered only if the test_expression evaluates to True. The body starts with indentation and the first unindented line marks the end. Python interprets any non-zero value as True. None and 0 are interpreted as False.	

Fig: operation of while loop

```
# Program to add natural numbers up to sum = 1+2+3+...+n
n = 10
# initialize sum and counter
sum = 0
i = 1
while i <= n:
    sum = sum + i
    i = i+1    # update counter
# print the sum
print("The sum is", sum)
```

Output:

```
The sum is 55
```

- **The range() function:**

  To loop through a set of code a specified number of times, we can use the range() function, The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

  We can also define the start, stop and step size as range(start, stop,step_size). step_size defaults to 1 if not provided.

```
# range(n) prints numbers 0 to n-1
# Outputs 0 to 5
for x in range(6):
    print(x)
# Outputs 2 to 5
for x in range(2, 6):
    print(x)
# Outputs 2, 5, 8, 11, 14, 17
for x in range(2, 20, 3):
    print(x)
```

- **The break Statement:**

  The break statement terminates the loop containing it. Control of the program flows to the statement immediately after the body of the loop.

  If the break statement is inside a nested loop (loop inside another loop), the break statement will terminate the innermost loop.

```
# Exit the loop when x is banana
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        break
    print(x)
```

Output:

apple

- **The continue Statement:**

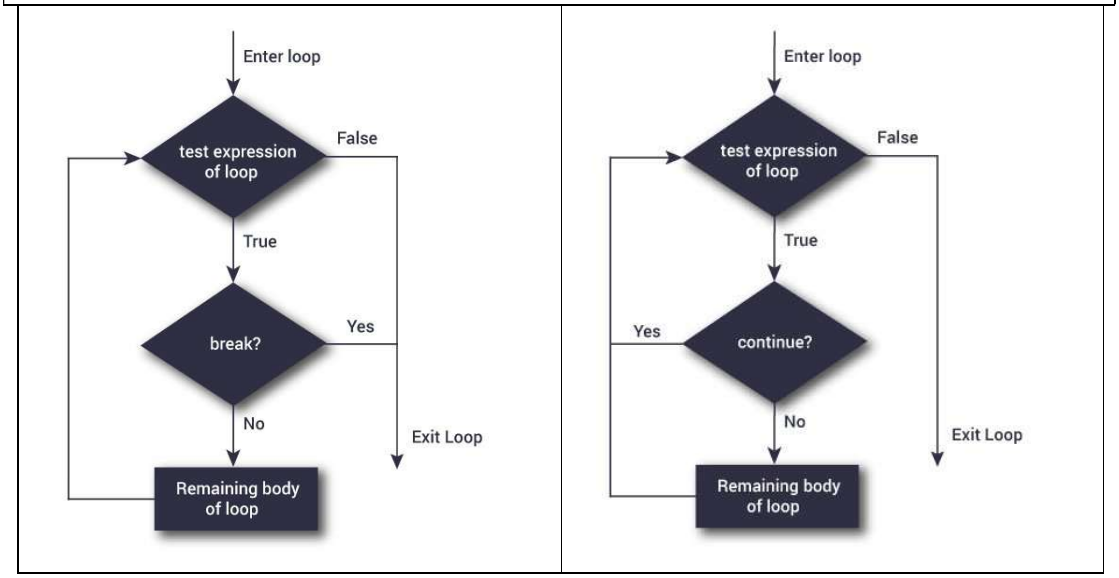
  The continue statement is used to skip the rest of the code inside a loop for the current iteration only. Loop does not terminate but continues on with the next iteration.

```
# Exit the loop when x is banana
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    if x == "banana":
        continue
    print(x)
```

Output:

apple

cherry



- **Loop with else (use with for and while loop):**

A for loop can have an optional else block as well. The else part is executed if the items in the sequence used in for loop exhausts.

The break keyword can be used to stop a for loop. In such cases, the else part is ignored. Hence, a for loop's else part runs if no break occurs.

```
digits = [0, 1, 5]
for i in digits:
    print(i)
else:
    print("No items left.")
```

Output:

0

1

5

No items left.

Strings….

- **Python Strings are immutable**

  In Python, strings are immutable. That means the characters of a string cannot be changed.

```
message = 'Hola Amigos'
message[0] = 'H'
print(message)
```

Output

TypeError: 'str' object does not support item assignment

However, we can assign the variable name to a new string.

```
message = 'Hola Amigos'
# assign new string to message variable
message = 'Hello Friends'
prints(message); # prints "Hello Friends"
```

- **Python Multiline String**

  We can also create a multiline string in Python. For this, we use triple double quotes """ or triple single quotes '''.

```
# multiline string
message = """
Never gonna give you up
Never gonna let you down
"""
print(message)
```

Output

```
Never gonna give you up
Never gonna let you down
```

- **String Operations:**

  There are many operations that can be performed with strings which makes it one of the most used data types in Python.

  1. Compare Two Strings

  We use the == operator to compare two strings. If two strings are equal, the operator returns True. Otherwise, it returns False.

```
str1 = "Hello, world!"
```

```
str2 = "I love Python."
str3 = "Hello, world!"
# compare str1 and str2
print(str1 == str2)
# compare str1 and str3
print(str1 == str3)
```

Output

```
False
True
```

In the above example,

str1 and str2 are not equal. Hence, the result is False.

str1 and str3 are equal. Hence, the result is True.

2. Join Two or More Strings

In Python, we can join (concatenate) two or more strings using the + operator.

```
greet = "Hello, "
name = "Jack"

# using + operator
result = greet + name
print(result)
```

Output

```
Hello, Jack
```

Run Code

3. Iterate Through a Python String

We can iterate through a string using a for loop.

```
greet = 'Hello'
# iterating through greet string
for letter in greet:
    print(letter)
```

Output

```
H
e
l
l
o
```

5. Python String Length

In Python, we use the len() method to find the length of a string.

```
greet = 'Hello'
# count length of greet string
print(len(greet))
```

Output

```
5
```

6. String Membership Test

We can test if a substring exists within a string or not, using the keyword in.

```
print('a' in 'program') # True
print('at' not in 'battle') False
```

7. Methods of Python String

Besides those mentioned above, there are various string methods present in Python. Here are some of those methods:

Methods	Description
upper()	converts the string to uppercase
lower()	converts the string to lowercase
partition()	returns a tuple
replace()	replaces substring inside
find()	returns the index of first occurrence of substring
rstrip()	removes trailing characters
split()	splits string from left
startswith()	checks if string starts with the specified string
isnumeric()	checks numeric characters
index()	returns index of substring

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 3

Part B (to be completed by students)

Looping and Strigs

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp3)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp3.pdf).
6. Upload the pdf on the web portal